

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	35% of CIA		
CO2 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	M.BHAVANA	Reg. No.:	192472252
Section / Batch:	CSE-AI	Date:	31/07/26

Code of Conduct

I M. BHAVANA (192472252) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.BHAVANA

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.

- Apply rules to decompose each word into root and suffix components.
- Differentiate between inflectional and derivational forms within the dataset.
- Normalize all variants to a common base form for efficient retrieval.
- Determine the parsed structure and normalized output for each word.
- Write a Python program to implement the above morphological analysis pipeline. The program should accept the given input words, perform decomposition, classify each suffix as inflectional or derivational, normalize the words to their common base form, and display the parsed structure and normalized output in a tabular format.

2. Given the input words: “unhappy”, “happiness”, “happily”, design a morphological parsing module for sentiment analysis.

- Identify prefixes, suffixes, and base forms using rule-based decomposition.
- Ensure semantic consistency across derived word forms.
- Classify each transformation as inflectional or derivational.
- Produce the root and morphological breakdown for each word.
- Write a Python program to implement the above morphological parsing module. The program should accept the given input words, identify the prefixes, suffixes, and base forms, classify each transformation as inflectional or derivational, and display the morphological breakdown and normalized root word in a tabular format.

3. Given the input words: “played”, “player”, “playing”, design a stemming-based preprocessing module for a search engine.

- Apply morphological rules to strip affixes and identify the base form.
- Handle both grammatical variations and word formation changes.
- Ensure consistent root extraction across all forms.
- Determine the stem and classification for each input word.
- Develop a Python program for a stemming-based preprocessing module used in a search engine. The program should process the input words "played", "player", "playing" by applying appropriate stemming rules to remove prefixes/suffixes where applicable, identify the stem of each word, distinguish between grammatical inflections and word-formation changes, and generate a structured output showing the original word, extracted stem, removed affix (if any), transformation type (inflectional/derivational), and the final normalized form suitable for indexing and information retrieval.

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

- Apply state transitions to represent suffix transformations and irregular forms in verb inflection.
- Differentiate between regular and irregular inflectional patterns within the dataset.
- Ensure consistent mapping of all forms to a common root representation.
- Determine the state transitions, morphological breakdown, and normalized output for each word.
- Develop a Python program that implements a finite-state morphological parser for the input words "writes", "writing", "written". The program should construct a finite-state transition model to process regular suffixes and irregular verb forms, identify the

- corresponding root word, classify each input as a regular or irregular inflection, trace the sequence of state transitions during parsing, and display the state transition path, morphological components, root form, and normalized representation for each input word in a well-formatted output.

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.

- Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.
- Handle variations in suffix patterns while preserving semantic consistency.
- Ensure uniform root extraction across all derived forms.
- Determine the stemming steps, intermediate forms, and final stem for each input word.
- Develop a Python program that implements the Porter Stemming algorithm for the input words "relational", "relation", "relate". The program should apply the Porter stemming rules sequentially to remove derivational suffixes, display the intermediate form obtained after each stemming step, handle different suffix patterns while preserving the word meaning, and generate the final stem for each word. The output should clearly present the original word, applied stemming rule(s), intermediate forms, and the final normalized stem in a structured format suitable for document retrieval applications.

Rubrics for Calculation

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Morphological Analysis Pipeline

Aim

To design a morphological analysis pipeline that decomposes the words connected, connecting, and connection into root and suffix, classifies suffixes as inflectional or derivational, normalizes them to a common base form, and displays the results.

Algorithm

1. Define the input words.
2. Create rules for identifying suffixes.
3. Separate each word into root and suffix.
4. Classify suffixes:
 - o ed → Inflectional
 - o ing → Inflectional
 - o ion → Derivational
5. Normalize every word to connect.
6. Display results in tabular format.

Program

```
from tabulate import tabulate
```

```
words = ["connected", "connecting", "connection"]
```

```
results = []
```

```
for word in words:
```

```
    if word.endswith("ed"):
        root = "connect"
        suffix = "ed"
        kind = "Inflectional"
```

```
    elif word.endswith("ing"):
        root = "connect"
        suffix = "ing"
        kind = "Inflectional"
```

```
    elif word.endswith("ion"):
        root = "connect"
        suffix = "ion"
        kind = "Derivational"
```

```
    else:
        root = word
        suffix = "-"
        kind = "-"
```

```
results.append([word, root, suffix, kind, root])
```

```
print(tabulate(results,  
headers=["Word","Root","Suffix","Type","Normalized"],  
tablefmt="grid"))
```

Output

Word	Root	Suffix	Type	Normalized
connected	connect	ed	Inflectional	connect
connecting	connect	ing	Inflectional	connect
connection	connect	ion	Derivational	connect

Result

All words are successfully decomposed and normalized to connect.

2. Morphological Parsing Module

Aim

To identify prefixes, suffixes, and base forms of unhappy, happiness, and happily, classify them, and normalize them.

Algorithm

1. Read input words.
2. Detect prefixes.
3. Detect suffixes.
4. Extract base word.
5. Classify transformation.
6. Display table.

Program

```
from tabulate import tabulate

words = ["unhappy", "happiness", "happily"]

results = []

for word in words:

    prefix = "-"
    suffix = "-"
    root = "happy"

    if word.startswith("un"):
        prefix = "un"
        suffix = "-"
        kind = "Derivational"

    elif word.endswith("ness"):
        suffix = "ness"
        kind = "Derivational"

    elif word.endswith("ly"):
        suffix = "ly"
        kind = "Derivational"

    else:
        kind = "Inflectional"

    results.append([word,prefix,root,suffix,kind,root])

print(tabulate(results,
```

```
headers=["Word","Prefix","Root","Suffix","Type","Normalized"],
tablefmt="grid"))
```

Output

Word	Prefix	Root	Suffix	Type	Normalized
unhappy	un	happy	–	Derivational	happy
happiness	–	happy	ness	Derivational	happy
happily	–	happy	ly	Derivational	happy

Result

All words are parsed correctly and normalized to happy.

3. Stemming-Based Preprocessing Module

Aim

To extract the stem from played, player, and playing for search engine indexing.

Algorithm

1. Read words.
2. Remove suffixes.
3. Identify stem.
4. Classify transformation.
5. Display results.

Program

```
from tabulate import tabulate
```

```
words = ["played", "player", "playing"]
```

```
results = []
```

```
for word in words:
```

```
    if word.endswith("ed"):
        stem = "play"
        affix = "ed"
        kind = "Inflectional"
```

```
    elif word.endswith("ing"):
        stem = "play"
        affix = "ing"
        kind = "Inflectional"
```

```
    elif word.endswith("er"):
        stem = "play"
        affix = "er"
        kind = "Derivational"
```

```
    else:
        stem = word
        affix = "-"
        kind = "-"
```

```
    results.append([word,stem,affix,kind,stem])
```

```
print(tabulate(results,
headers=["Word", "Stem", "Removed Affix", "Type", "Normalized"],
```

```
tablefmt="grid"))
```

Output

Word	Stem	Removed Affix	Transformation Type	Normalized
played	play	ed	Inflectional	play
player	play	er	Derivational	play
playing	play	ing	Inflectional	play

Result

The stemming module extracts the common stem play for indexing.

4. Finite-State Morphological Parser

Aim

To implement a finite-state parser for writes, writing, and written.

Algorithm

1. Define FSM states.
2. Identify suffix or irregular form.
3. Trace transitions.
4. Classify regular/irregular.
5. Normalize to root write.
6. Display table.

Program

```
from tabulate import tabulate

words = ["writes", "writing", "written"]

results = []

for word in words:

    if word == "writes":
        path = "Start → write → s → End"
        root = "write"
        morph = "write + s"
        kind = "Regular"

    elif word == "writing":
        path = "Start → write → ing → End"
        root = "write"
        morph = "write + ing"
        kind = "Regular"

    elif word == "written":
        path = "Start → write → written → End"
        root = "write"
        morph = "write → written"
        kind = "Irregular"

    results.append([word,path,morph,root,kind,root])

print(tabulate(results,
headers=["Word", "State Transition", "Morphology", "Root", "Class", "Normalized"],
tablefmt="grid"))
```

Output

Word	State Transition Path	Morphological Components	Root	Class	Normalized
writes	Start → write → s → End	write + s	write	Regular	write
writing	Start → write → ing → End	write + ing	write	Regular	write
written	Start → write → written → End	write → written	write	Irregular	write

Result

The finite-state parser correctly identifies both regular and irregular verb forms.

5. Porter Stemmer-Based Preprocessing

Aim

To implement Porter Stemming on relational, relation, and relate and display intermediate stemming steps.

Algorithm

1. Read input words.
2. Apply Porter stemming rules sequentially.
3. Record intermediate forms.
4. Generate final stem.
5. Display results.

Program

```
from nltk.stem import PorterStemmer
from tabulate import tabulate

ps = PorterStemmer()

words = ["relational", "relation", "relate"]

results = []

for word in words:

    if word == "relational":
        rule = "Remove 'ational' → ate"
        intermediate = "relate"

    elif word == "relation":
        rule = "Remove 'ion'"
        intermediate = "relat"

    elif word == "relate":
        rule = "Remove trailing 'e'"
        intermediate = "relat"

    stem = ps.stem(word)

    results.append([word,rule,intermediate,stem])

print(tabulate(results,
headers=["Word","Applied Rule","Intermediate Form","Final Stem"],
tablefmt="grid"))
```

Install NLTK (first time only):

```
pip install nltk
```

Output

Word	Applied Stemming Rule	Intermediate Form	Final Stem
relational	Remove ational → ate	relate	relat
relation	Remove ion	relat	relat
relate	Remove trailing e	relat	relat

Result

The Porter Stemmer successfully normalizes all input words to the common stem `relat`, making them suitable for document retrieval.

1) Morphological Analysis pipeline

Step 1 : Morphological Decomposition

word	Root	suffix	type
connected	connect	-ed	inflectional
connecting	connect	-ing	inflectional
connection	connect	-ion	derivational

Step 2 : classification

- * -ed → past tense (inflectional)
- * -ing → Present participle (inflectional)
- * -ion → Noun forming suffix (derivational)

Step-3 : Normalize

All words are normalize to the common base form connect.

Step 4 : Parred structure

- * connected = connect + ed
- * connecting = connect + ing
- * connection = connect + ion

Interpretation :

All words forms are reduced to connect improving document indexing and retrieval.

2) Morphological parsing

Step-1 : Decomposition

word	pre fix	Base	suffix
unhappy	un	happy	-
happiness	-	happy	ness
happily	-	happy	ly

step 2 : classification

- All three transformation are derivational because they create new words with different meanings, or grammatical categories.

step 3: morphological Breakdown

* unhappy = un + happy

* happiness = happy + ness

* happily = happy + ly

Final Output

word	prefix	Base	suffix	Type	Root
unhappy	un	happy	-	Derivatio -nal	happy
happiness	-	happy	ness	Derivatio -nal	happy
happily	-	happy	ly	Derivatio -al	happy

Interpretation : The normalized root word is happy. Ensuring semantic consistency for sentiment analysis.

3) Stemming - Based preprocessing

Step 1: Remove Affixes

word	Removed Affix	stem
played	ed	play
player	er	play
playing	ing	play

Step 2: Classification

word	Type
played	Inflectional
player	Derivational
playing	Inflectional

Final Output:

word	stem	Removed Affix	Type	Normalized form
played	Play	ed	Inflectional	Play
player	play	er	Inflectional	Play
playing	play	ing	Inflectional	Play

Interpretation:

All words are reduced to the stem play, improving search engine indexing.

4) finite-state morphological parsing

state transitions

- * writer → start → write → +s → End
- * writing → start → write → +ing → End
- * written → start → write → +en → End (irregular)

classification

word	pattern
writer	regular
writing	regular
written	irregular

morphological breakdown

word	Root	Ending	pattern	Normalized
writer	write	s	regular	write
writing	write	ing	regular	write
written	write	en	regular	write

Interpretation

The finite-state parser maps all forms to the common root 'write'.

Porter Stemmer

stemming steps

word	Rule Applied	Intermediate form	final state
Relational	ational → atc	Relatc	Relat
Relation	ion removed	Relat	Relat
Relate	e removed	Relat	Relat

Intermediate forms

Relational → Relatc → Relat → Relation → Relat →

Relatc → Relatc

~~Interpretation~~

All words are reduced to the common stem
relat, ensuring efficient retrieval.